

KinectFusion

Ignacio Dassori Tom Schimansky Aaron Schulze Mats Grobe

03801592

03807186

03757764

03800203

Technical University of Munich
Garching, Germany

{ignacio.dassori, tom.schimansky, aaron.schulze, mats.grobe}@tum.de

Abstract

We propose a full re-implementation of KinectFusion, the seminal real-time dense SLAM system presented by Newcombe et al. (2011) [2]. KinectFusion addresses the challenge of real-time, dense 3D reconstruction of indoor environments by leveraging depth measurements from a Kinect sensor. Other 3D reconstruction methods, such as structure-from-motion and multi-view stereo, typically rely on offline processing and global optimization over large image sets. Instead, a KinectFusion pipeline enables real-time, online reconstruction by directly fusing depth measurements into a global, consistent volumetric scene representation. To achieve real-time performance, the individual pipeline components are optimized through CPU-side parallelism using OpenMP and GPU acceleration with CUDA. The application of our pipeline on standard a RGB-D benchmark dataset shows stable pose estimation and reconstruction for the majority of the trajectory.

1. Motivation and Idea

Real-time 3D reconstruction is an open research problem that has many applications in the fields of robotics, computer vision and autonomous vehicles. Reconstructing real-world scenes while estimating camera motion and fusing incoming depth data into a coherent geometric model allows applications such as indoor scanning, navigation, object reconstruction, and mixed-reality environments. Typical approaches include continuously recording a scene with a moving sensor to integrate and accumulate depth measurements into a single model which is iteratively refined.

Designing such systems is an intricate balance between reconstruction fidelity, speed, computational resources and scene assumptions. The types of sensors and cameras that are deployed in this context can also vary from monocular vision cameras to RGB-D sensors, multi-camera rigs, structured light systems, and time-of-flight depth cameras.

This project is the implementation of a functional version of KinectFusion, a dense SLAM (Simultaneous Localization and Mapping) system that reconstructs a 3D surface model in real-time as the scene is being recorded from a commodity depth sensor such as the Microsoft Kinect. The system tracks the camera pose in 3D using projective Iterative Closest Point (ICP), integrates live depth frames into a Truncated Signed Distance Function (TSDF) volume and continuously raycasts the TSDF to generate synthetic depth maps for pose tracking. The implementation is a valuable demonstration of a real-time dense SLAM pipeline.

2. Related Work

Early work on real-time 3D scene reconstruction was typically constrained by reliance on monocular RGB cameras, requiring multi-view geometry and extensive camera motion to infer depth. A typical example is the Parallel Tracking and Mapping (PTAM) system [1]. Its main contribution lies in demonstrating the feasibility of real-time bundle adjustment, which was previously considered computationally too expensive. Using only a single RGB camera, PTAM achieves stable, real-time tracking and mapping by optimizing over a selected subset of keyframe features. However, this filtering results in a sparse scene representation and limits the density of the reconstructed map. Subsequent work therefore tried to address this limitation, which led development of a Dense Tracking and Mapping (DTAM) system [3]. DTAM extends monocular SLAM toward dense scene reconstruction by jointly estimating camera motion and a dense depth map across all image pixels. The introduction of depth-based sensors, in particular the Kinect sensor, eventually enabled access to consumer-level depth measurement systems. While PTAM and DTAM demonstrated the potential for real-time scene reconstruction using monocular RGB input, they relied on estimating scene depth from image data. The availability of direct depth measurements removed this dependency, paving the way for robust, real-time dense reconstruction systems such as KinectFusion.

3. Method

Our work closely follows the system and notation as introduced by the original KinectFusion paper by [2].

3.1. Depth Map Conversion and Pre-processing

For an in-coming RGB-D frame k , a bilateral filter is applied to the raw depth measurement to reduce sensor noise at each image pixel location $\mathbf{u} = (u, v)^T$. This results in the filtered depth map D_k . Each filtered depth value is then back-projected into the sensor coordinate frame using the camera intrinsic matrix K , producing a vertex map V_k . A corresponding normal map N_k is obtained from the cross product of neighboring vertex differences:

$$N_k(u) = \nu[(V_k(u+1, v) - V_k(u, v)) \times (V_k(u, v+1) - V_k(u, v))] \quad (1)$$

where $\nu[x] = x/\|x\|_2$ applies normalization. The vertex and normal maps serve as geometric input for the subsequent ICP-based sensor pose estimation.

3.2. Sensor Pose Estimation

Point-to-plane ICP is used to iteratively update the estimate of the current pose of the camera $\tilde{T}_{g,k}^z$ by computing incremental transforms $\tilde{T}_{inc}^z$ throughout z iterations. The small angle assumption is used for the incremental transform:

$$\tilde{T}_{inc}^z = [\tilde{R}^z | \tilde{t}^z] = \begin{bmatrix} 1 & \alpha & -\gamma & t_x \\ -\alpha & 1 & \beta & t_y \\ \gamma & -\beta & 1 & t_z \end{bmatrix}. \quad (2)$$

To solve for the parameters $\mathbf{x} = (\beta, \gamma, \alpha, t_x, t_y, t_z)^T \in \mathbb{R}^6$ we first find correspondences between frames, compute their point-to-plane errors, and solve a linearized least squares optimization problem.

We run ICP using a coarse-to-fine strategy. A pyramid of increasing input resolution is built, with the first iterations using low resolution, and the last iterations using the complete original maps.

3.2.1. Projective Data Association

The localization of the camera is done through a frame-to-model tracking method. Correspondences between the current vertex and normal maps (V_k, N_k) and the model predictions of the previous frame $(\hat{V}_{k-1}, \hat{N}_{k-1})$ are computed with projective data association. To compute correspondences, first we perspective project the current vertex map onto the previous camera's image plane using the current estimate of the frame-to-frame transform $\tilde{T}_{g,k}^z$:

$$\hat{\mathbf{u}} = \pi(K\tilde{T}_{k-1,k}^z \dot{V}_k(\mathbf{u})), \quad (3)$$

$$\tilde{T}_{k-1,k}^z = T_{g,k-1}^{-1} \tilde{T}_{g,k}^z, \quad (4)$$

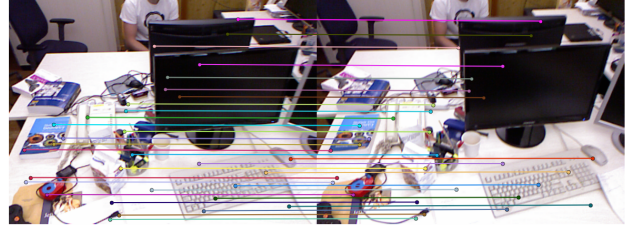


Figure 1. Projective data association correspondences.

where $\dot{V}_k(\mathbf{u})$ is the vertex map in homogeneous coordinates. On every ICP call $\tilde{T}_{g,k}^{z=0}$ is initialized with the previous frame pose. The thresholds ε_d and ε_θ are introduced to reject correspondences with large euclidean distance and angle deviation. Additionally, correspondences are discarded for invalid depth values ($z < 0$) and for pixel coordinates $\mathbf{u}, \hat{\mathbf{u}}$ outside of the map range. The set of valid correspondences is given by $\{V_k(\mathbf{u}), \hat{V}_{k-1}(\hat{\mathbf{u}}) \mid \Omega(\mathbf{u}) \neq \text{null}\}$, where $\Omega(\mathbf{u}) = \text{null}$ if any of the previous conditions holds.

3.2.2. Pose Optimization

The global point-to-plane energy is computed using the found correspondences:

$$\mathbf{E}(T_{g,k}) = \sum_{\Omega_k(\mathbf{u}) \neq \text{null}} \left\| (T_{g,k} \dot{V}_k(\mathbf{u}) - \hat{V}_{k-1}^g(\hat{\mathbf{u}}))^T \hat{N}_{k-1}^g(\hat{\mathbf{u}}) \right\|_2 \quad (5)$$

$$E = (T_{g,k} \dot{V}_k(\mathbf{u}) - \hat{V}_{k-1}^g(\hat{\mathbf{u}}))^T \hat{N}_{k-1}^g(\hat{\mathbf{u}}) \quad (6)$$

With the small angle assumption and the parameter vector $\mathbf{x}$, we linearize the error function using

$$\tilde{T}_{g,k}^z \dot{V}_k(\mathbf{u}) = \tilde{R}^z \tilde{V}_k^g(\mathbf{u}) + \tilde{t}^z = \mathbf{G}(\mathbf{u})\mathbf{x} + \tilde{V}_k^g(\mathbf{u}), \quad (7)$$

where

$$\tilde{V}_k^g(\mathbf{u}) = \tilde{T}_{g,k}^{z-1} \dot{V}_k^g(\mathbf{u}), \quad (8)$$

$$\mathbf{G}(\mathbf{u}) = \left[[\tilde{V}_k^g(\mathbf{u})]_{\times} \mid \mathbf{I}_{3 \times 3} \right]. \quad (9)$$

By replacing 7 on 6 we get the linearized energy term and our linear least squares optimization problem:

$$\min_{\mathbf{x} \in \mathbb{R}^6} \sum_{\Omega_k(\mathbf{u}) \neq \text{null}} \|E\|_2^2 \quad (10)$$

Computing the derivative with respect to $\mathbf{x}$ and setting it to zero results in the 6×6 linear system

$$\sum_{\Omega_k(\mathbf{u}) \neq \text{null}} (\mathbf{A}^T \mathbf{A}) \mathbf{x} = \sum \mathbf{A}^T \mathbf{b}, \quad (11)$$

where $\mathbf{A} \in \mathbb{R}^{1 \times 6}$ and $\mathbf{b} \in \mathbb{R}$ are computed per correspondence pair as follows:

$$\mathbf{A} = \hat{N}_{k-1}^g(\hat{\mathbf{u}})^T \mathbf{G}(\mathbf{u}), \quad (12)$$

$$\mathbf{b} = \hat{\mathbf{N}}_{k-1}^g(\hat{\mathbf{u}})^T \left(\hat{\mathbf{V}}_{k-1}^g(\hat{\mathbf{u}}) - \tilde{\mathbf{V}}_k^g(\mathbf{u}) \right). \quad (13)$$

We sum together the contributions from all correspondences and solve the 6×6 for $\mathbf{x}$ using the Cholesky decomposition. With $\mathbf{x}$ we can reconstruct the incremental transform and update our estimate of the current camera pose, iterating this process over z iterations:

$$\tilde{\mathbf{T}}_{g,k}^z = \tilde{\mathbf{T}}_{inc}^z \tilde{\mathbf{T}}_{g,k}^{z-1} \quad (14)$$

3.2.3. CPU and GPU Implementations (CUDA)

Given the fact that correspondence pair finding and computation of 12 and 13 are done on a per-pixel basis, these parts of the pipeline can be massively optimized through parallelization. The only exception to this is the summing of all contributions to form the linear system. On the CPU implementation, we use OpenMP for multi-core processing, using *openmp critical* to reduce partial results to global $(\mathbf{A}^T \mathbf{A})$ and $(\mathbf{A}^T \mathbf{b})$ matrices.

The GPU implementation uses a custom CUDA kernel to determine correspondences and local $(\mathbf{A}^T \mathbf{A})$ and $(\mathbf{A}^T \mathbf{b})$ matrices. The reduction of the partial results is performed in two stages. First, each thread accumulates per-point contributions to $(\mathbf{A}^T \mathbf{A})$, $(\mathbf{A}^T \mathbf{b})$, the squared residual, and the inlier count. For each block, the per-thread accumulations are reduced using warp-level shuffle operations, followed by a shared-memory reduction across warps, producing one partial result per block. In an additional kernel launch, the per-block partial results are reduced using the same strategy, leading to the global results.

3.3. Volumetric Mapping/Integration

3.3.1. Volumetric Representation

To represent the global scene geometry, a Truncated Signed Distance Function (TSDF) is accumulated within a fixed 3D voxel grid. Each voxel stores the truncated signed distance $F_k(p)$, a weight $W_k(p)$ representing the confidence of the measurement and an RGB color value for visualization.

3.3.2. Projective Data Association

For every incoming depth map D_k with an associated camera pose $T_{g,k}$ estimated by the Pose Optimization module, a integration update is performed. We iterate over the voxel grid, where each voxel center p_g in global coordinates is transformed into the camera coordinate frame:

$$p_c = T_{g,k}^{-1} p_g \quad (15)$$

The point is then projected onto the image plane using the camera intrinsic matrix K to find the corresponding pixel coordinates $\mathbf{u} = (u, v)^T$. If $\mathbf{u}$ lies within the image bounds, the measured depth $d_{meas} = D_k(\mathbf{u})$ is sampled.

3.3.3. TSDF Update and Fusion

The signed distance value (*sdf*) is computed as the difference between the measured depth and the distance of the voxel from the camera center along the optical axis (z_c):

$$sdf = d_{meas} - z_c \quad (16)$$

A truncation threshold μ is applied to focus the reconstruction on the surface interface. The fused TSDF value $F_k(p)$ is updated using a weighted running average:

$$F_k(p) = \frac{W_{k-1}(p)F_{k-1}(p) + \Psi(sdf)}{W_{k-1}(p) + 1} \quad (17)$$

where Ψ is the truncation function clamping the value between $[-1, 1]$. The weight $W_k(p)$ is incremented up to a maximum threshold W_{max} .

3.3.4. CPU and GPU Implementations (CUDA)

The CPU version utilizes OpenMP "collapse" directives to parallelize the outermost z and y grid traversal loops across CPU cores, leaving the inner x loop for each thread to process. It allocates the grid as a flat, contiguous `std::vector` in memory. This linear memory layout maximizes cache locality and prefetching efficiency.

The GPU version is implemented in CUDA, unrolling the loops into a 3D kernel launch where each thread processes a block of $8 \times 8 \times 8$ voxels, resulting in 512 threads for a default gridsize of 256. The memory transfer overhead is minimized by keeping the volume in VRAM, which is also shared with the Raycaster module.

3.4. Surface Prediction

3.4.1. Ray Generation

For each pixel (u, v) in the target image of width W and height H , a ray is cast from the camera center into the world. Given the intrinsic matrix K and the current estimate $T_{g,k} = [R|t]$, we define the ray origin $\mathbf{o} = t$. The ray direction in world coordinates is calculated as:

$$d_{cam} = \begin{bmatrix} (u - c_x)/f_x \\ (v - c_y)/f_y \\ 1 \end{bmatrix}, \quad d_{world} = \frac{R \cdot d_{cam}}{\|R \cdot d_{cam}\|} \quad (18)$$

3.4.2. Trilinear Interpolation

Because the TSDF volume is a discrete grid, sampling the field at an arbitrary continuous point p requires trilinear interpolation to prevent jagged artifacts. Given a point p and its surrounding eight neighbor voxels $V_{i,j,k}$, the value $F(p)$ is computed as a weighted average:

$$F(p) = \sum_{i,j,k \in \{0,1\}} V_{i,j,k} \cdot w_i(x)w_j(y)w_k(z) \quad (19)$$

where $w_0(d) = 1 - (d - \lfloor d \rfloor)$ and $w_1(d) = d - \lfloor d \rfloor$. This ensures that the field is continuous, allowing for smooth normal estimation and sub-voxel surface accuracy.

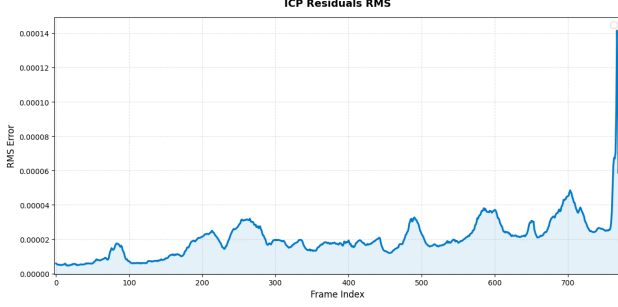


Figure 2. Point-to-plane residuals RMS.

3.4.3. Ray Marching and Surface Estimation

The algorithm steps along the ray with a step size Δt . A surface is detected when a zero-crossing is found in the TSDF values ($F_k(p_{i-1}) > 0$ and $F_k(p_i) < 0$). To find the precise surface point p_s , linear interpolation is applied:

$$p_s = p_{i-1} + \Delta t \left(\frac{F_k(p_{i-1})}{F_k(p_{i-1}) - F_k(p_i)} \right) \quad (20)$$

3.4.4. Normal Estimation

The surface normal $\mathbf{n}$ at point $\mathbf{p}$ is the normalized gradient of the TSDF field computed using central differences:

$$\nabla F(\mathbf{p}) \approx \begin{bmatrix} F(\mathbf{p} + \delta\mathbf{x}) - F(\mathbf{p} - \delta\mathbf{x}) \\ F(\mathbf{p} + \delta\mathbf{y}) - F(\mathbf{p} - \delta\mathbf{y}) \\ F(\mathbf{p} + \delta\mathbf{z}) - F(\mathbf{p} - \delta\mathbf{z}) \end{bmatrix} \quad (21)$$

If color is stored in the volume, the predicted pixel color is retrieved by performing a final trilinear interpolation of the RGB values at p_s .

3.4.5. CPU and GPU Implementations (CUDA)

The CPU version utilizes OpenMP for multi-core acceleration. The GPU implementation is written in CUDA, where threads are mapped to pixels using a 2D block structure (16×16). Voxel data is accessed via a global memory pointer to the TSDF volume. The resulting vertices and normals are transformed back into the camera coordinate system (using R^T) before being stored.

4. Results

$$\hat{p}_t = g(z_t) \quad (22)$$

$$\text{MSE}_{\text{geom}} = \frac{1}{N} \sum_{t=1}^N \|\hat{p}_t - p_t\|^2 \quad (23)$$

$$\text{MSE}_{\text{pred}} = \frac{1}{N} \sum_{t=1}^N \|\hat{p}_{t+k} - p_{t+k}\|^2$$

We evaluate the performance of our Volume, Raycaster and ICP modules using both CPU (OpenMP) and GPU (CUDA) backends. Experiments are conducted on a 24-core Intel i9 CPU (32 threads) and RTX 5080 GPU. As a

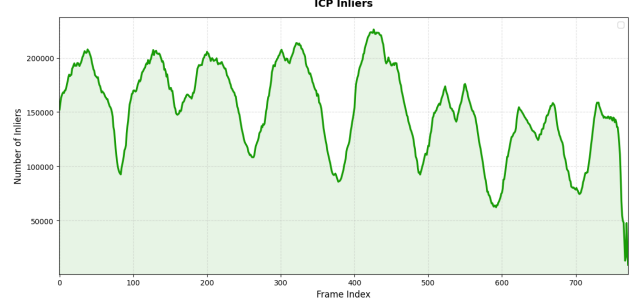


Figure 3. Number of valid correspondences per frame.

data source, we use the 'freiburg1_xyz' sequence from the TUM RGB-D dataset [4]. For our experiments we use a cubical sized volume of size 1024 with 0.01m voxel size. Furthermore, we only time the parts of the pipeline we implemented ourselves and exclude the virtual sensor and bilateral filter from our measurements.

Component	Backend	Avg (ms)	Speedup	FPS
Raycast	Single-CPU	130132.00	1×	—
	OpenMP	9959.15	13×	—
	CUDA	2.03	64262×	—
Integration	Single-CPU	131789.00	1×	—
	OpenMP	8552.05	15×	—
	CUDA	3.57	36915×	—
ICP	Single-CPU	14852.40	1×	—
	OpenMP	1117.43	13×	—
	CUDA	15.68	947×	—
Total System	Single-CPU	276773.40	1×	0.0036
	OpenMP	19628.63	14×	0.051
	CUDA	21.28	13006×	46.99

Table 1. Performance scaling across backends (dataloading, bilateral filtering excluded)

The results summarized in Table 1 reflect the final optimized state of our implementation, achieving a total throughput of 46.99 FPS on the GPU.

5. Analysis

Throughput Analysis The throughput jump from 0.051 FPS for OpenMP to 46.99 FPS for CUDA is critical for the KinectFusion pipeline. The OpenMP implementation is bottlenecked primarily by the raycasting stage. Offloading these operations to the GPU enables a Raycasting duration of 2ms compared to 1 s before, allowing for real-time performance. Conversely, with the CUDA implementation, the ICP now becomes the most expensive component with about 16ms. This is caused by the parallel nature of the Raycasting and Volume Integration while the ICP includes global reductions leading to computational overhead.

ICP Stability Analysis We analyze the robustness of our tracking by monitoring the Root Mean Square (RMS) error of the point-to-plane residuals and the number of valid correspondences (inliers) over the freiburg1_xyz sequence.

As shown in Fig. 2, the RMS error remains consistently low ($<4\text{mm}$) for the majority of the trajectory, indicating stable convergence and high-quality surface alignment, although steadily accumulating as the sequence progresses. However, a distinct error spike with a simultaneous drop in correspondences is constantly observed at the last 20 frames of the sequence, independent of the chosen ICP parameters.

We find two possible explanations for this behavior: We observe, by inspecting the frames, that the camera trajectory remains almost fixed during the last 20 frames, which could lead to small residual values and a rank-deficient $A^T A$ matrix. Alternatively, the issue might be caused by small inaccuracies in the TSDF volume caused by incorrect poses found by ICP during the sequence. The steady accumulation of errors, combined with a static camera at the end, could lead to repeated false correspondences and, in turn, to the quick divergence of the pose. This hypothesis is backed by the results of Figure 2, as well as the almost constant number of inliers found in the last handful of frames right before the collapse in Figure 3.

CUDA Implementation While our CUDA implementation is working reliably on physical PC setups we experience a memory allocation bug when running inside a remote virtual machine, which might be a limitation of services such as Paperspace or caused by an imperfect implementation on our side.

6. Conclusion

We successfully re-implemented the KinectFusion pipeline and benchmarked a CPU-based OpenMP approach against a GPU-accelerated CUDA backend. Our results confirm that while CPU parallelism offers a 14x speedup, it remains insufficient for dense volumetric mapping in real-time. In contrast, the CUDA implementation achieves real-time performance, representing a 900x improvement over OpenMP. General tracking accuracy was high and we noticed instabilities only for trajectory edge-cases.

References

- [1] Georg Klein and David Murray. Parallel tracking and mapping for small ar workspaces. In *2007 6th IEEE and ACM International Symposium on Mixed and Augmented Reality*, pages 225–234, 2007. 1
- [2] Richard A. Newcombe, Shahram Izadi, Otmar Hilliges, David Molyneaux, David Kim, Andrew J. Davison, Pushmeet Kohli, Jamie Shotton, Steve Hodges, and Andrew William Fitzgibbon. Kinectfusion: Real-time dense surface mapping and

tracking. *2011 10th IEEE International Symposium on Mixed and Augmented Reality*, pages 127–136, 2011. 1, 2

- [3] Richard A. Newcombe, Steven J. Lovegrove, and Andrew J. Davison. Dtam: Dense tracking and mapping in real-time. In *2011 International Conference on Computer Vision*, pages 2320–2327, 2011. 1
- [4] J. Sturm, N. Engelhard, F. Endres, W. Burgard, and D. Cremers. A benchmark for the evaluation of rgb-d slam systems. In *Proc. of the International Conference on Intelligent Robot Systems (IROS)*, 2012. 4